



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

SISTEMAS DISTRIBUIDOS

Laboratorio S08

Saga Orquestada y Circuit Breaker

Nombre del alumno:	Yukioayax Canek Gabriel Hernández
Profesor:	Oscar Manuel Ruiz Hurtado
Grupo:	2
Semestre en curso:	2027-1

Ciudad Universitaria, CDMX

Índice

1. Introducción **2**

2. Parte 0: Reproducción del problema **2**

3. Parte 1: Saga Orquestada **3**

4. Parte 2: Circuit Breaker **4**

5. ¿Qué pasa si la compensación también falla? **5**

6. Conclusiones **5**

1 Introducción

El presente reporte documenta el desarrollo del Laboratorio S08, cuyo objetivo central es resolver el problema de consistencia de datos en arquitecturas de microservicios, ya que cuando una operación cruza múltiples servicios (como sucede en una transferencia bancaria), la anotación `@Transactional` del monolito deja de ser suficiente, lo que puede resultar en la pérdida de dinero si un servicio falla a la mitad del proceso; por ello, se implementó una **Saga Orquestada** con lógica de compensación y un **Circuit Breaker** para manejar la resiliencia ante caídas inesperadas.

2 Parte 0: Reproducción del problema

Antes de arreglar el código se levanto los servicios y creando dos cuentas de prueba: Ana con 1000 pesos y Beto con 500 pesos.

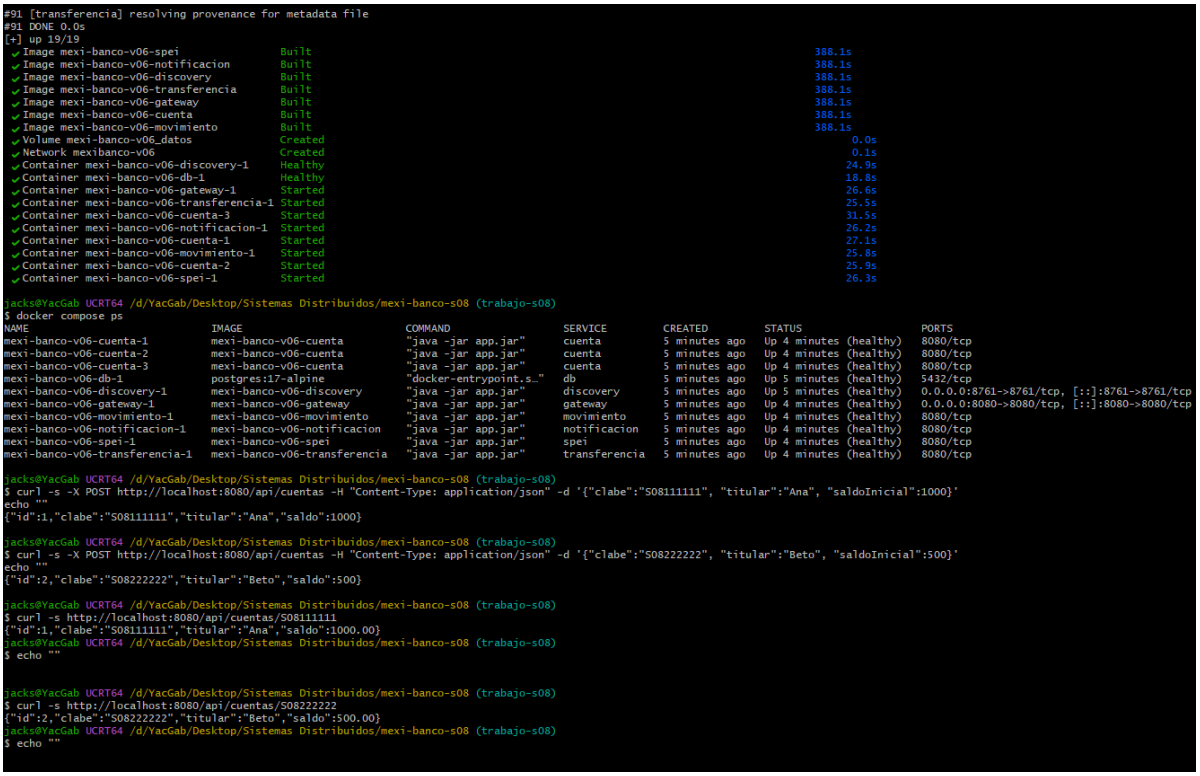


Figura 1: Creación de cuentas y verificación de saldos iniciales.

Posteriormente, se configuró una pausa de 20 segundos en el servicio de transferencia y se lanzó una petición; durante esa pausa, se apagó el servicio de `cuenta`, simulando un fallo de red o una caída del servidor.

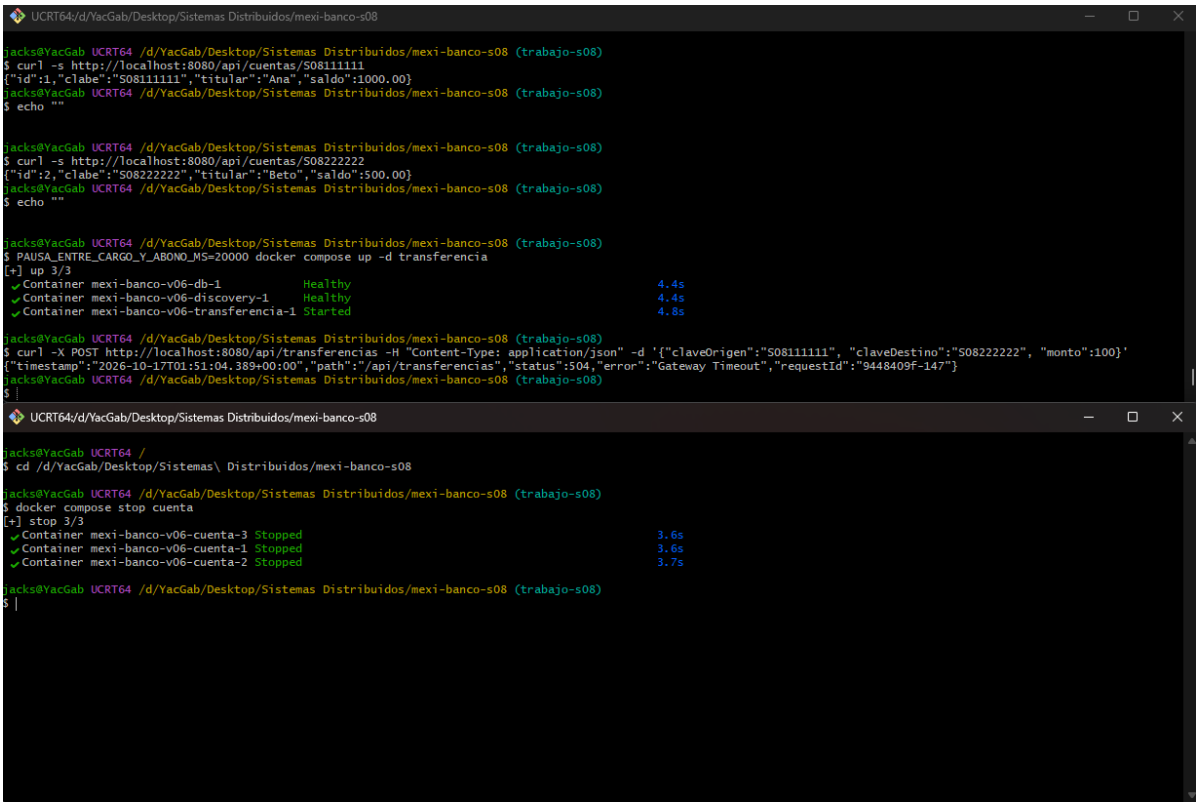


Figura 2: Apagado del servicio `cuenta` durante la pausa de la transferencia.

Al volver a encender el servicio y consultar los saldos, se comprobó que el dinero salió de la cuenta de Ana (quedando en 900) pero no llegó a la de Beto (quedándose en 500), es decir, el dinero se quedó en el aire.

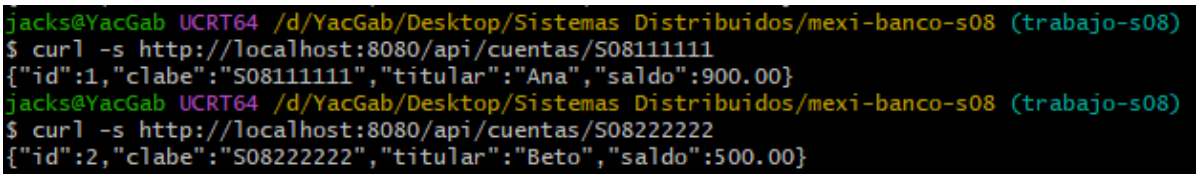


Figura 3: Pérdida del dinero: el saldo de Ana disminuyó, pero el de Beto no aumentó.

3 Parte 1: Saga Orquestada

Para solucionar el problema anterior, se reescribió `TransferenciaService.java` para que actúe como un orquestador, añadiendo estados a la entidad `Transferencia` (INICIADA, CARGADA, ABONADA, COMPLETADA, COMPENSADA, etc.); de esta forma, si el abono falla después de que el cargo se aplicó, el sistema captura la excepción y ejecuta una **compensación** para devolver el dinero a la cuenta de origen. Si la compensación tiene éxito, la transferencia queda en estado `COMPENSADA`, y si falla, queda en `COMPENSACION_PENDIENTE`, lo que indica que requiere un reintento.

TransferenciaService.java — Lógica de Compensación

```
} catch (Exception e) { cbCuenta.registrarFallo(); if
(transferencia.getEstado() == EstadoTransferencia.CARGADA ||
transferencia.getEstado() == EstadoTransferencia.ABONADA) {
try { cuentas.abonar(claveOrigen, monto); // Devolver a Ana
transferencia.setEstado(EstadoTransferencia.COMPENSADA); } catch (Exception
exCompensacion) { transferencia.setEstado(EstadoTransferencia.COMPENSACION_PENDI
} } }
```

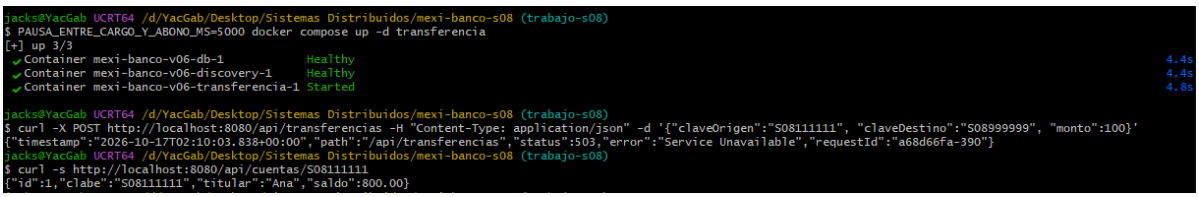


Figura 4: Verificación del saldo de Ana tras la ejecución de la saga.

4 Parte 2: Circuit Breaker

Se implementó un Circuit Breaker hecho a mano (`MiCircuitBreaker.java`) para evitar que el sistema siga enviando peticiones a un servicio caído; este componente cuenta con tres estados: CERRADO (flujo normal), ABIERTO (rechaza peticiones tras 3 fallos) y SEMIABIERTO (deja pasar una petición de prueba tras un tiempo determinado).
Para probarlo, se apagó el servicio de `cuenta` y se realizaron varias peticiones, por lo que el sistema respondió con `503 Service Unavailable` y el log del balanceador mostró que no había servidores disponibles; además, las transferencias que sí llegaron a registrarse se marcaron como `FALLIDA`, evitando así que el sistema colapsara por completo.

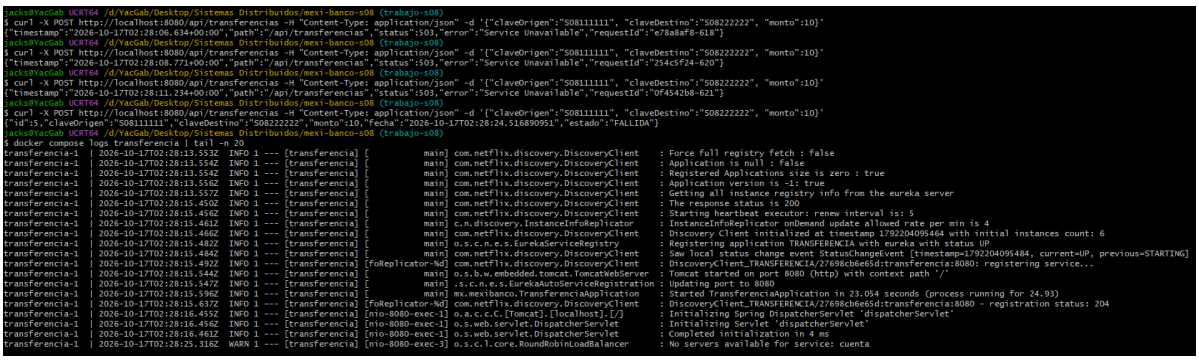


Figura 5: Logs del sistema mostrando el rechazo de peticiones y la marca de FALLIDA al no encontrar el servicio de cuenta.

5 ¿Qué pasa si la compensación también falla?

Si el servicio de `cuenta` está caído, es muy probable que la compensación (devolver el dinero) también falle; el código maneja este escenario dejando la transferencia en estado `COMPENSACION_PENDIENTE`, lo que significa que el sistema es consciente de que el dinero está en el limbo, para solucionarlo, se requeriría un proceso en segundo plano que reintente la compensación periódicamente, ya que es crucial que esta compensación sea **idempotente** para evitar errores catastróficos como devolver el dinero dos veces.

6 Conclusiones

La saga orquestada y el circuit breaker son patrones esenciales en sistemas distribuidos; la saga permite mantener la consistencia eventual de los datos sin necesidad de transacciones globales, mientras que el circuit breaker protege al sistema de fallos en cascada cuando un servicio dependiente deja de responder, por lo que implementar estos patrones resulta vital para construir sistemas robustos y tolerantes a fallos.